How each flow works

Short step lists for every flow in the system. Names in `code` are the real functions and tables.

Jump to (the part of the system that runs it is in brackets):

1. Create a job (API)
2. Publish tasks to the queue (API and scheduler)
3. Worker runs a GENERATE task (worker)
4. Apply a result (scheduler)
5. Combine (planner and worker)
6. Final task and completion (planner, worker, scheduler)
7. Retry after a failure (worker and scheduler)
8. Duplicate and late results (scheduler)
9. Cancel (API)
10. Backup copy for a slow task (scheduler)
11. Dead worker (scheduler and broker)
12. Change the number of workers (API and worker manager)
13. The 500 ms tick (scheduler)
14. Start-up and reconnect (all)
15. Progress and the UI (API and browser)

One job from start to end (F = 12, one possible run)

- Job creation makes 3 GENERATE tasks: files `[0..4]`, `[5..9]`, `[10, 11]`.
- Workers generate 12 raw files. Each is an item with count 1.
- 5 items wait, so the planner creates a COMBINE task. The result is a partial with count 5. This happens twice.
- The last 2 raw files and the 2 partials are waiting (4 items). All files exist and nothing else is running, so the planner creates the **final** task.
- The final task adds them, divides by 12 and writes `result.csv`. Count = 12 = F, so the job is `COMPLETED`.

---

1. Create a job

- `POST /jobs` with `{F, C}`. The API checks 1 < F < 100,000 and 1 < C < 1,000,000, otherwise `400`.
- One transaction:
- Insert the job (`PENDING`).
- Split the file indexes into groups of 5 (`splitIntoChunks`).
- Insert one GENERATE task per group (`PENDING`).

- Log "job created".
- After the commit, publish tasks to the queue (flow 2).
- The API answers `202 {jobId}` and the work continues in the background.

---

2. Publish tasks to the queue (the dispatcher)

- Runs after a job is created, after each result, and on every tick.
- It keeps the queue short: target depth is `max(2, 2 × W)`. Free slots = target − tasks already waiting.
- It picks `PENDING` tasks in turns between jobs: the 1st task of every job, then the 2nd of every job, and so on. This is `lockPendingTasksFairly`.
- The picked rows are locked with `SKIP LOCKED`, so a second caller skips them instead of waiting.
- For each task:
- Publish to RabbitMQ and wait for the broker's confirm.
- Then mark it `QUEUED` and count the attempt.
- A crash between the two steps only publishes the task again, which is safe.
- Only one run happens at a time inside a process.

---

3. Worker runs a GENERATE task

- The worker takes one message at a time (`prefetch = 1`).
- It blocks `SIGTERM` and marks itself `BUSY`.
- It checks the job in the database. If the job is cancelled or failed, it reports `SKIPPED` and stops.
- It rejects a task with more than 5 inputs.
- For each file index:
- The seed is derived from the job id and the file index, so a retry writes the same data.
- It creates C random values (float32) and writes `raw/<index>.npy`. The write goes to a temp name and is renamed.
- The output is `{path, count: 1}`.
- It publishes the result (status, outputs, duration) and waits for the confirm.
- Then it acks the task, adds 1 to `tasks_done`, unblocks `SIGTERM`, and goes back to `IDLE`.

---

4. Apply a result (scheduler)

- Results are read one at a time, in arrival order.
- A malformed message is logged and dropped.
- One database transaction:
- Lock the task, then the job.
- Task already `DONE`: log a duplicate and stop (flow 8).
- Job cancelled or failed, or the worker said `SKIPPED`: mark the task `FAILED` and discard the work.

- Worker said `FAILED`: retry flow (flow 7).
- Success: mark the task `DONE` with the worker and duration. The first success moves the job `PENDING` → `RUNNING`.
- Save the outputs as `READY` items. For a GENERATE task, add the new files to `files_generated`.
- Run the planner (flow 5), then log the event.
- After the commit: ack the message and publish pending tasks (flow 2).
- If the apply fails, the message is requeued and retried after 1 s.

---

5. Combine

The planner (`planCombineTasks`) runs for one job inside a transaction that holds the job lock. Rules, in order:

1. 5 or more items waiting: take 5 and create a combine task. Repeat while 5 or more remain.
2. Fewer than 5 left, all files generated, nothing else unfinished, 2 or more waiting: take all and create the **final** task.
3. 2 or more waiting, a worker is idle and the queue is empty: take all and create a smaller combine.
4. 1 item left, all done: a final task that only divides.
5. Anything else: wait for the next result.

Creating the task

- Insert a COMBINE task (`PENDING`) whose payload lists the inputs: item id, path and count.
- Set those items to `USED` and `used_by = the task`, in the same transaction. No other task can take them.

The worker

- Starts a running total of zeros (float64) and a total count.
- For each input, one at a time: load the file, add it position by position, add its count.
- Writes the total to `partials/<taskId>.npy`.
- Reports `{path, count: total count}`.

Back in the scheduler

- The output is saved as a new `READY` item. It can be combined again later, at any level.

---

6. Final task and completion

- The planner creates the final task when all files exist, nothing else is running, and fewer than 5 items are left.
- The worker does the same addition, then **divides every position by the total count**.
- It writes `result.npy` and `result.csv` (one value per line) and reports the count.
- The scheduler checks the count:
- Count = F: job `COMPLETED`, `result_path` saved.
- Count ≠ F: job `FAILED` with "count mismatch". This is a safety net and shouldn't happen.

- The user downloads the CSV with `GET /jobs/:id/result` (`409` until the job is completed).

---

7. Retry after a failure

- A worker reports `FAILED` when the task throws an error, or has more than 5 inputs.
- Attempts are counted each time a task is published.
- Fewer than 3 attempts: the task goes back to `PENDING` with its error, and the dispatcher publishes it again.
- 3 attempts used: the task is `FAILED` and so is the job, with a message such as "task failed 3 times: ".
- Retried tasks write the same data to the same path, so nothing is duplicated.

---

8. Duplicate and late results

- A task that is already `DONE`: the result is ignored and "duplicate result ignored" is logged.
- `items.path` is unique and the insert uses `ON CONFLICT DO NOTHING`, so an item is never stored twice.
- A result for a cancelled or failed job: the task is marked `FAILED` ("job not active") and no items are created.
- An unknown task id is ignored.
- Where duplicates come from: a redelivered message, a backup copy, or a crash before an ack.

---

9. Cancel

- `DELETE /jobs/:id`. The job row is locked.
- Unknown id: `404`. Job already finished: `409`.
- One transaction:
- Job becomes `CANCELLED`.
- Tasks not yet published become `FAILED` ("cancelled").
- Log "job cancelled".
- Then publish pending tasks, so the freed queue slots go to other jobs.
- Tasks already on the queue or running:
- A worker that hasn't started yet sees the cancelled job and reports `SKIPPED`.
- A worker that is mid-task finishes. Its result is discarded (flow 8).
- Files already written stay on disk. There is no cleanup yet.

---

10. Backup copy for a slow task

- Checked on every tick, only if a worker is idle and the queue is empty.
- Candidates: tasks held by a `BUSY` worker, in a `RUNNING` job, with no backup yet. Longest running first, at most one per idle worker.

- The median duration is taken over the last 50 finished tasks of the same type. No history means no backup.
- A task is slow if it has run longer than `max(1 s, 2 × median)`.
- For a slow task:
- Publish a duplicate message (marked as a backup).
- Set `backup_sent = true`, so a task gets at most one backup.
- Log "backup copy of … on workers-N".
- The first result wins. The second hits flow 8 and is ignored.

---

11. Dead worker

- Each worker updates `last_seen` every 2 s, from a separate thread.
- On every tick, a worker whose heartbeat is older than 10 s is marked `DEAD` and its current task is cleared. This is logged once.
- Its task was never acknowledged, so RabbitMQ puts it back on the queue when the connection closes. Another worker takes it.
- The worker manager restarts a crashed process. The new worker registers again.
- A worker that was only stalled starts heartbeating again, and flips from `DEAD` back to `IDLE`.
- Not covered: a frozen worker that keeps its connection open. RabbitMQ waits for its own heartbeat timeout before redelivering.

---

12. Change the number of workers

- `PUT /workers {count}`. The API accepts an integer from 0 to 32 and saves it as `desired_workers` in the `settings` table. It also logs "worker count set to N".
- The worker manager reads that setting every 2 s:
- Forget processes that have exited.
- **Too few:** start new processes, reusing the lowest free index (`workers-0`, `workers-1`, …). The slowdown factor comes from the index.
- **Too many:** send `SIGTERM` to the surplus processes, highest index first.
- A worker that gets `SIGTERM`:
- Finishes its current task. The signal is held back until the task is acknowledged.
- Shows as `DRAINING`, then exits and removes its row.
- An idle worker notices the request within about 1 s.

---

13. The 500 ms tick

Runs in the scheduler, never two at once. Each tick:

1. Mark dead workers (flow 11).
2. Take a snapshot: idle workers, busy workers, tasks waiting in the queue, desired workers.

3. For every `RUNNING` job with 2 or more waiting items, lock the job and run the planner. This is what lets rule 3 fire without a new result.
4. Send backup copies (flow 10).
5. Publish pending tasks (flow 2).

A failed tick is logged and the next one runs normally.

---

14. Start-up and reconnect

`docker compose up`

- `postgres` starts and creates the tables from `db/schema.sql`.
- `rabbitmq` starts. The others wait until both are healthy.
- `api` serves the REST API and the UI.
- `scheduler` starts consuming results and runs the tick.
- `workers` starts the manager. It saves the default worker count once, then starts the worker processes. Each worker registers in the `workers` table and starts its heartbeat.

Lost connection to RabbitMQ

- The scheduler's channel closes, and it reconnects after 2 s and consumes again.
- Messages are durable, so tasks and results survive.
- Workers lose their connection too. The manager restarts any worker process that exits.

Lost connection to Postgres

- A failed result is requeued. A failed tick is logged and retried 500 ms later.

---

15. Progress and the UI

Progress

- `(files generated + merges done) / (2F − 1)`.
- A combine of n inputs counts as n − 1 merges.
- Capped at 99% until the job is `COMPLETED`, then 100%.

The UI

- Plain HTML, CSS and JavaScript. The page is picked by the URL hash.
- Each page asks the API once per second, for example `/stats`, `/jobs`, `/jobs/:id`, `/workers`.
- The activity log asks only for new lines: `/events?after=<last id>`, or `?job=<id>` for one job.
- Buttons call the same API: New job (`POST /jobs`), Cancel (`DELETE /jobs/:id`), Apply (`PUT /workers`), Download (`GET /jobs/:id/result`).